

Comunicación ROS 2 ↔ Pixhawk

Flujo de datos desde el nodo Python hasta los motores del F550

Versión 2 — Mayo 2026 | Proyecto: dron_control_sistema | Firmware: PX4 |
Conexión: USB /dev/ttyACM0

Sección	Contenido
1	Vista general de la pila de comunicación
2	MAVLink: el protocolo binario del dron
3	MAVROS: el puente ROS 2 ↔ MAVLink
4	Transformación de coordenadas ENU ↔ NED
5	Flujo completo: de la tecla al motor
6	Pipeline interno PX4
7	Modo OFFBOARD y watchdog de PX4
8	Servicios ROS 2: petición-respuesta
9	Parámetros de la conexión serie
10	Comportamiento ante pérdida de conexión
11	Conexión USB real: WSL2 y usbipd-win (nuevo)
12	Mejoras al nodo mavros_node (nuevo)
13	Verificación round-trip: [PX4 ACK] (nuevo)
14	Problema QoS y solución (nuevo)
15	Estado actual del proyecto (nuevo)

1. Vista general de la pila

La cadena completa desde que el usuario presiona una tecla hasta que el motor ajusta sus RPM pasa por cuatro capas diferenciadas:

Capa	Qué hace	Tecnología
Aplicación	Genera órdenes (menú, misión)	Python / ROS 2
Transporte ROS 2	Pub/sub y servicios	DDS (rmw_fastrtps)
Puente	Convierte msgs ROS2 ↔ MAVLink	MAVROS
Protocolo dron	Telemetría y control serie	MAVLink v2

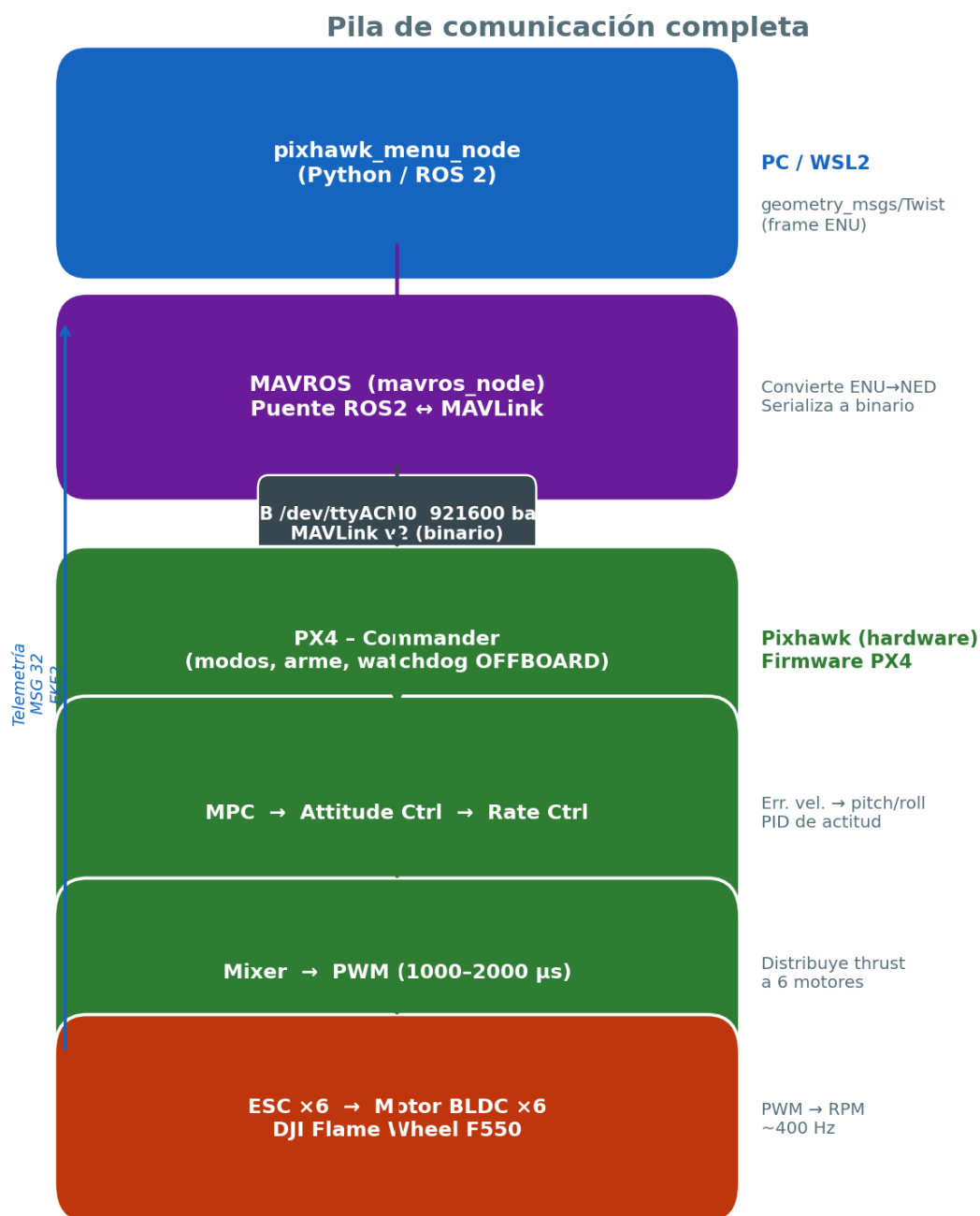


Figura 1 — Pila de comunicación completa: del nodo Python al motor BLDC.

2. MAVLink: el idioma del dron

MAVLink es un protocolo binario ligero diseñado para vehículos autónomos. Cada mensaje incluye un identificador de tipo (Message ID), los IDs del sistema emisor/receptor, el payload con los datos y un CRC de verificación.

Formato de trama MAVLink v2



Ejemplo: SET_POSITION_TARGET_LOCAL_NED (ID 84) → payload contiene vx=0.3, vy=0, vz=0 en NED + máscara de tipo

Figura 2 — Estructura de una trama MAVLink v2 con payload de velocidad.

Mensajes clave en este proyecto:

ID	Nombre	Dirección	Contenido
76	COMMAND_LONG	PC → Pixhawk	Armar, cambiar modo, despegar
84	SET_POSITION_TARGET_LOCAL_NEDPC → Pixhawk		Setpoint velocidad (vx, vy, vz)
0	HEARTBEAT	Bidireccional	Estado del sistema (1 Hz)
32	LOCAL_POSITION_NED	Pixhawk → PC	Posición estimada por EKF2 (30 Hz)
77	COMMAND_ACK	Pixhawk → PC	Confirmación de COMMAND_LONG

3. MAVROS: el puente ROS 2 ↔ MAVLink

MAVROS es un nodo ROS 2 externo que mantiene abierta la conexión serie con el Pixhawk. Actúa como traductor bidireccional:



Figura 3 — Grafo de nodos ROS 2: topics (azul) y servicios (verde).

Topic / Servicio ROS 2	Tipo de mensaje	Mensaje MAVLink
<code>/mavros/setpoint_velocity/cmd_vel_unstamped</code>	<code>geometry_msgs/Twist</code>	SET_POSITION_TARGET_LOCAL_NED (84)
<code>/mavros/state</code>	<code>mavros_msgs/State</code>	HEARTBEAT (0) + SYS_STATUS (1)
<code>/mavros/local_position/pose</code>	<code>geometry_msgs/PoseStamped</code>	LOCAL_POSITION_NED (32)
<code>/mavros/cmd/arming (srv)</code>	<code>mavros_msgs/CommandBool</code>	COMMAND_LONG → ARM_DISARM
<code>/mavros/set_mode (srv)</code>	<code>mavros_msgs/SetMode</code>	COMMAND_LONG → DO_SET_MODE
<code>/mavros/cmd/takeoff (srv)</code>	<code>mavros_msgs/CommandTOL</code>	COMMAND_LONG → NAV_TAKEOFF
<code>/mavros/setpoint_raw/target_local</code>	<code>mavros_msgs/PositionTarget</code>	POSITION_TARGET_LOCAL_NED (85) — eco del setpoint

4. Transformación de coordenadas ENU ↔ NED

Este es el punto más importante y la fuente de errores más común. ROS 2 y MAVLink/PX4 usan marcos de referencia distintos. MAVROS aplica la conversión automáticamente, pero el programador debe saber en qué marco está escribiendo.

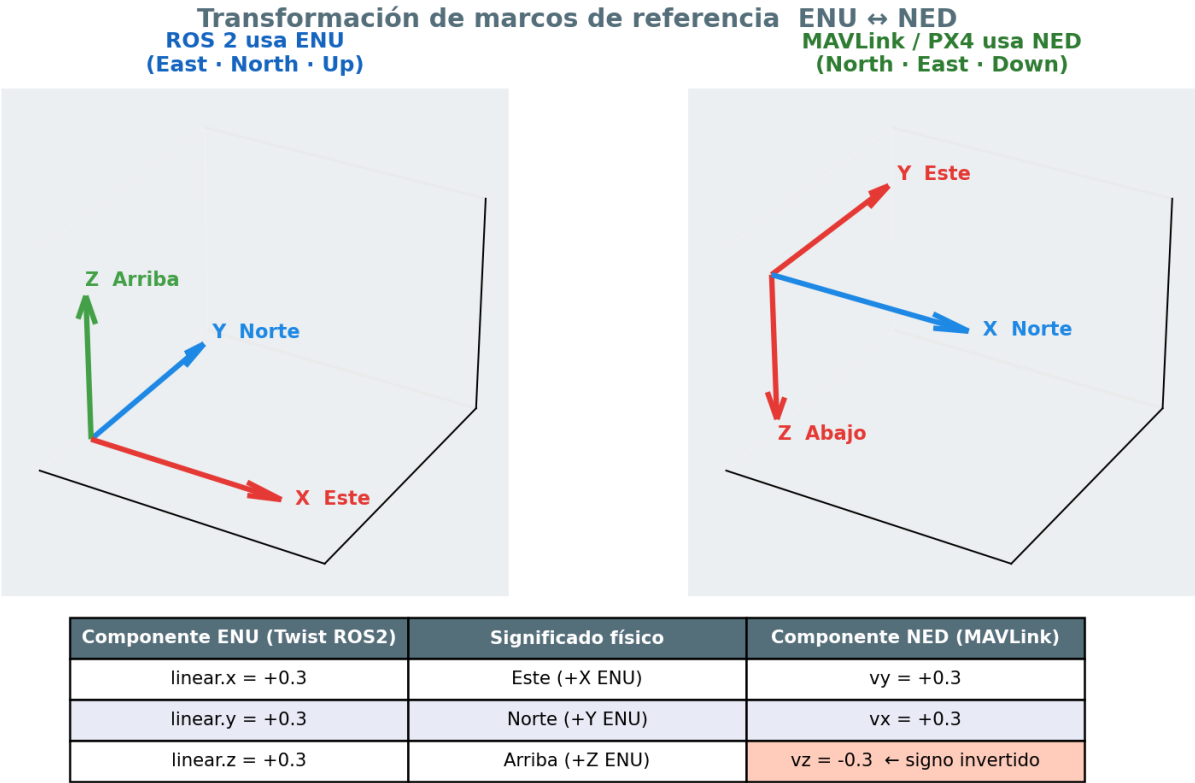


Figura 4 — Comparación de marcos ENU (ROS 2) y NED (MAVLink/PX4). El eje Z se invierte.

Regla de conversión que aplica MAVROS internamente:

$$v_{x_{NED}} = linear.y_{ENU} \quad v_{y_{NED}} = linear.x_{ENU} \quad v_{z_{NED}} = -linear.z_{ENU}$$

Ejemplo: pulsar [w] en el menú llama a `_enviar_vel(0.0, +0.3, 0.0)`, que pone `linear.y = +0.3` (Norte en ENU). MAVROS lo convierte a `vx=+0.3` m/s en NED, lo que hace que el dron se mueva hacia el Norte.

5. Flujo completo: de la tecla al motor

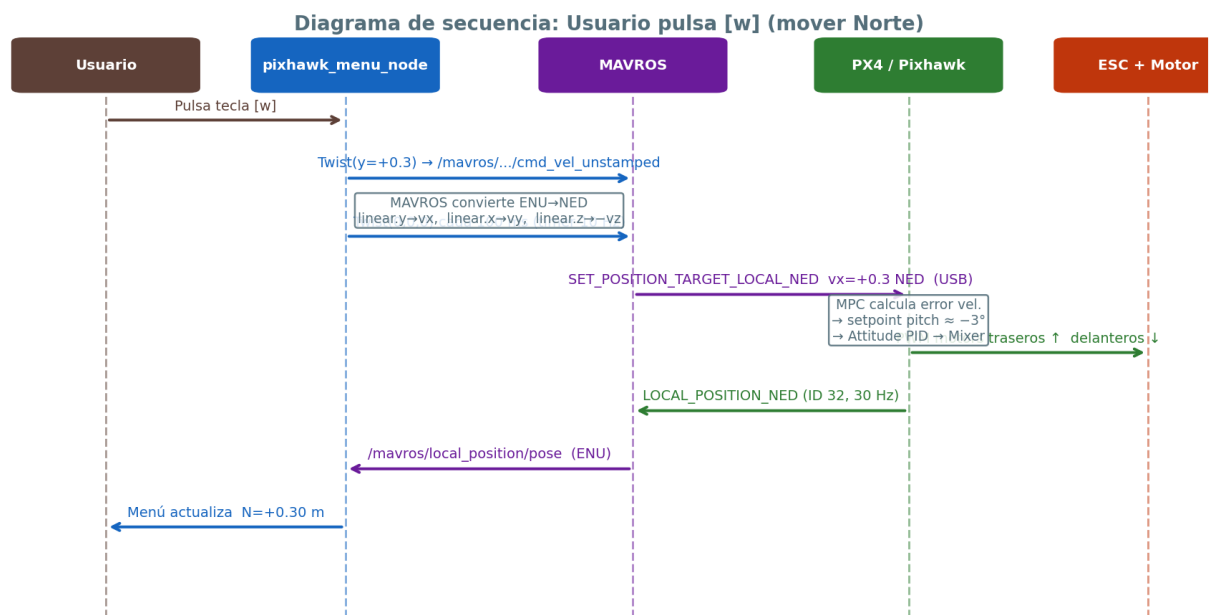


Figura 5 — Diagrama de secuencia: ciclo completo desde la pulsación de tecla hasta la respuesta del motor y la actualización de posición.

Resumen de pasos numerados:

1. El hilo de menú lee la tecla [w] y llama a `_enviar_vel(0.0, 0.3, 0.0)`.
2. El timer ROS 2 (10 Hz) publica el Twist al topic `/mavros/setpoint_velocity/cmd_vel_unstamped`.
3. MAVROS convierte ENU→NED y serializa el mensaje MAVLink ID 84 por el puerto serie USB a 921 600 baud.
4. PX4 recibe el mensaje, valida el CRC y lo publica en el bus interno uORB.
5. El MPC calcula el error de velocidad y genera un setpoint de pitch ($\sim -3^\circ$).
6. El controlador de actitud usa un PID para alcanzar ese pitch; el mixer distribuye entre los 6 motores.
7. Los ESC ajustan las RPM en milisegundos vía PWM.
8. El EKF2 fusiona IMU+GPS y publica `LOCAL_POSITION_NED` (ID 32) a 30 Hz.
9. MAVROS convierte NED→ENU y publica en `/mavros/local_position/pose`.
10. El nodo actualiza la posición mostrada en el menú.

6. Pipeline interno PX4

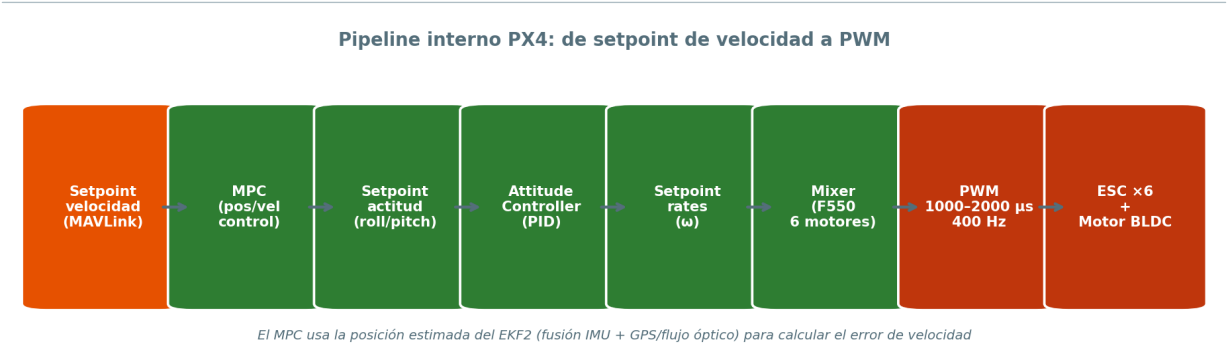


Figura 6 — Cadena de control interna de PX4: del setpoint de velocidad a la señal PWM.

El MPC (Multicopter Position Controller) necesita una estimación de posición/velocidad precisa. Sin GPS ni flujo óptico, no puede operar en modo OFFBOARD con comandos de velocidad. Para pruebas en interiores se suele usar un sistema de captura de movimiento (mocap) o T265.

7. Modo OFFBOARD: el watchdog de PX4

PX4 tiene un watchdog interno: si no llegan setpoints externos en **500 ms**, abandona OFFBOARD y entra en HOLD. Por eso el nodo publica a 10 Hz incluso cuando el dron está quieto (Twist con todos los campos a cero):

```
# pixhawk_menu_node.py – línea 51 self.create_timer(1.0 / TASA_HZ, self._timer_vel) # 10 Hz #
línea 79-80 if time.time() > self._activo_hasta: self._cmd_vel = Twist() # setpoint cero al
expirar la duración
```

Secuencia de arranque obligatoria para PX4:

Paso	Acción	Por qué
1	Arrancar pixhawk_menu_node	Inicia la publicación de setpoints a 10 Hz
2	Esperar ~2 s	PX4 verifica que ya hay setpoints en cola
3	[3] Modo OFFBOARD	PX4 acepta el cambio porque ya ve setpoints
4	[1] Armar	Los motores se activan
5	[4] Despegar	Sube a la altitud indicada
6	[w/s/a/d/r/f]	Envía velocidad durante 1 s, luego vuelve a cero

8. Servicios ROS 2: petición-respuesta

A diferencia de los topics (one-way), los servicios son síncronos (req/resp). El nodo usa `call_async` para no bloquear el executor de ROS 2 mientras espera la confirmación del Pixhawk (que puede tardar hasta ~500 ms):

```
# _armar() – línea 116 fut = self._cli_armar.call_async(req) fut.add_done_callback( lambda f:
self.get_logger().info( f"ARMAR: {'OK' if f.result().success else 'FALLO'}"))
```

El flujo interno es:

1. El nodo envía **CommandBool.Request(value=True)** al servicio.
2. MAVROS empaqueta un **COMMAND_LONG (ID 76)** con MAV_CMD_COMPONENT_ARM_DISARM y param1=1.0.
3. Pixhawk ejecuta los prearm checks (nivel batería, calibración IMU, etc.).
4. Responde con **COMMAND_ACK (ID 77)**: result=ACCEPTED o DENIED.
5. MAVROS traduce la respuesta a **CommandBool.Response(success=True/False)**.

9. Parámetros de la conexión serie

Parámetro	Valor	Observación
Puerto	/dev/ttyACM0	USB virtual CDC-ACM (como un Arduino)
Baudrate	921 600 bps	Máxima velocidad soportada por el USB virtual
Bits datos	8	Estándar
Paridad	Ninguna	—
Bits de stop	1	—
Control de flujo	Ninguno	—
Tiempo/trama	~0.2 ms	Para un mensaje de 20 bytes a 921 600 bps

Comando para arrancar MAVROS con PX4 (ejecutar antes del nodo menú):

```
ros2 run mavros mavros_node \ --ros-args \ -p fcu_url:=serial:///dev/ttyACM0:921600 \ -p
gcs_url:=''
```

10. Comportamiento ante pérdida de conexión

Evento	Reacción PX4	Parámetro relevante
Sin setpoints OFFBOARD > 500 ms	Sale de OFFBOARD → HOLD (flota en posición actual)	COM_RC_OVERRIDE
Sin HEARTBEAT del GCS > 3 s	Puede activar acción de failsafe	COM_DL_LOSS_T
Pérdida total de MAVLink	Aterrizar / RTL / mantener según configuración	NAV_DLL_ACT

Nota de seguridad: siempre tener a mano el mando RC con mayor prioridad que OFFBOARD para recuperar el control manualmente ante cualquier fallo del software.

11. Conexión USB real: WSL2 y usbipd-win

El Pixhawk se conecta por USB al PC Windows como **COM5**. WSL2 es una máquina virtual Linux que por defecto no puede ver dispositivos USB del host. Para reenviar el puerto al kernel Linux se usa **usbipd-win**, un driver de kernel de Microsoft.

Cadena USB: Pixhawk → Windows COM5 → usbipd-win → WSL2

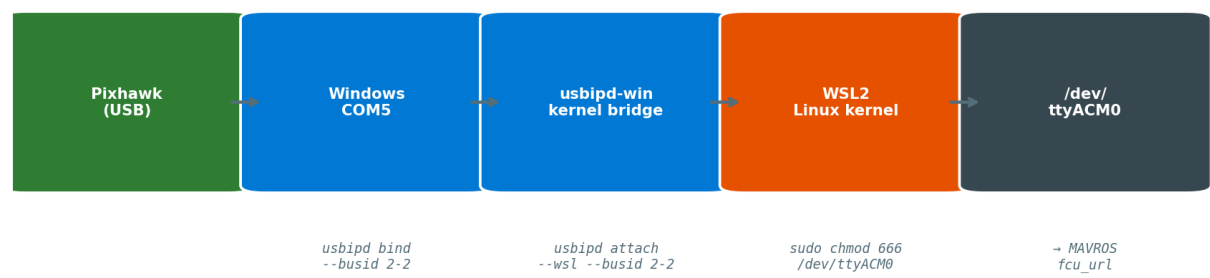


Figura 7 — Cadena completa de reenvío USB desde el Pixhawk hasta MAVROS en WSL2.

Pasos de configuración (una vez por sesión):

Dónde	Comando	Efecto
PowerShell (Admin)	winget install usbipd	Instala usbipd-win (solo la primera vez)
PowerShell (Admin)	usbipd list	Muestra BUSID del Pixhawk (VID 3185:0038)
PowerShell (Admin)	usbipd bind --busid 2-2	Marca el dispositivo para ser compartido
PowerShell (Admin)	usbipd attach --wsl --busid 2-2	Conecta el dispositivo a WSL2
WSL2	ls /dev/ttyACM*	Confirma que aparece /dev/ttyACM0
WSL2	sudo chmod 666 /dev/ttyACM0	Da permisos de lectura/escritura sin sudo
WSL2	ros2 launch mavros px4.launch fcu_url:=/dev/ttyACM0	Conecta MAVROS al Pixhawk

Señal de éxito en MAVROS: [mavros.mavros]: CON: Got HEARTBEAT, connected. FCU: PX4 Autopilot

12. Mejoras al nodo mavros_node

El `mavros_node.py` original solo publicaba en `/drone/cmd_vel` (el simulador) y leía posición de `/drone/pose`. Para conectar con el Pixhawk real se añadieron tres capacidades:

Qué se añadió	Por qué
Publisher → <code>/mavros/setpoint_velocity/cmd_vel_unstamped</code>	Comando que MAVROS escucha para reenviar al Pixhawk
Subscriber ← <code>/mavros/local_position/pose</code> (<code>PoseStamped</code>)	Posición real del Pixhawk vía EKF2; tiene prioridad sobre el simulador (<code>pos_source</code>)
Subscriber ← <code>/mavros/state</code> (<code>mavros_msgs/State</code>)	Monitorea si MAVROS está conectado; avisa si el comando puede no llegar al Pixhawk
Log enriquecido con <code>pos_source</code> , <code>mode</code> , <code>armed</code>	Permite verificar en tiempo real si la fuente de posición es el simulador o el Pixhawk

Ejemplo de log cuando MAVROS está conectado y hay target activo:

```
[INFO] [mavros] target=(2.40,1.50,1.00) pos=(0.12,0.08,0.00) vel=(+2.28,+1.42,+1.00)
modo=OFFBOARD armado=False
```

El prefijo `[mavros]` indica que la posición proviene del Pixhawk real, no del simulador.

13. Verificación round-trip: [PX4 ACK]

Para confirmar que un comando llega físicamente al Pixhawk se añadió en `pixhawk_menu_node.py` una suscripción al topic `/mavros/setpoint_raw/target_local`. Este topic es el eco que PX4 emite vía **POSITION_TARGET_LOCAL_NED** (MAVLink ID 85) cada vez que recibe y procesa un setpoint de velocidad.

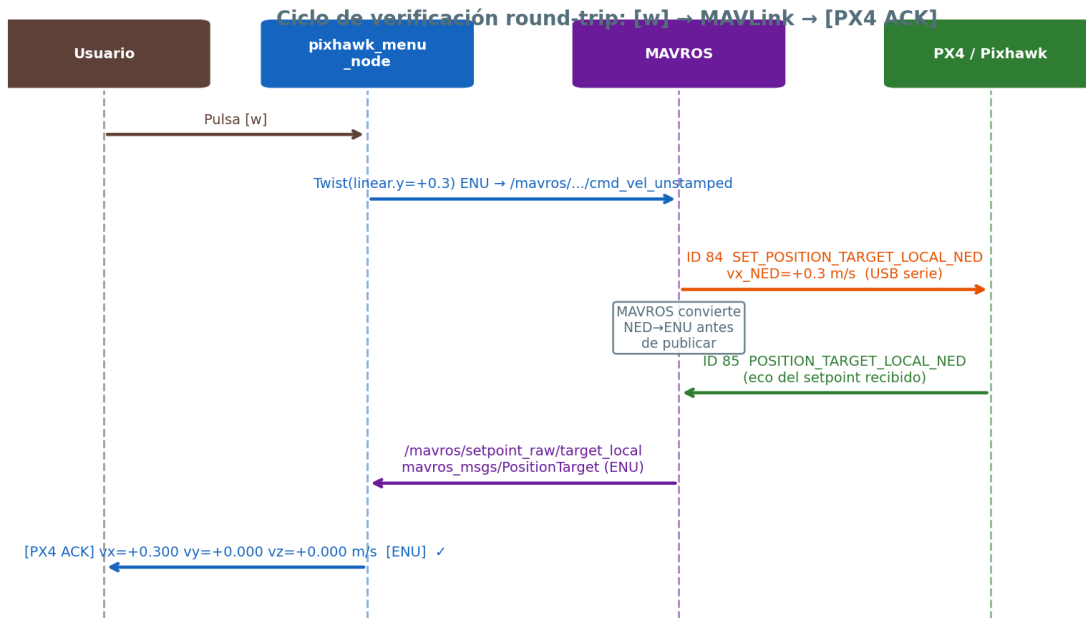


Figura 8 — Ciclo completo de verificación: desde la pulsación de [w] hasta el mensaje [PX4 ACK] que confirma el eco del Pixhawk.

Código clave en `pixhawk_menu_node.py`:

```
from mavros_msgs.msg import PositionTarget, State # en __init__: self.create_subscription(
PositionTarget, '/mavros/setpoint_raw/target_local', self._cb_ack_pixhawk, _qos) # callback:
def _cb_ack_pixhawk(self, msg): vx, vy, vz = msg.velocity.x, msg.velocity.y, msg.velocity.z
if abs(vx) < 0.01 and abs(vy) < 0.01 and abs(vz) < 0.01: return print(f'[PX4 ACK] vx={vx:+.3f}
vy={vy:+.3f} vz={vz:+.3f} m/s [ENU] ✓')
```

14. Problema QoS y solución

Al conectar el Pixhawk real se detectó que el menú mostraba **Modo: DESCONOCIDO** aunque MAVROS estuviera conectado. El log mostraba:

```
[WARN] New publisher discovered on topic /mavros/local_position/pose, offering incompatible QoS. No messages will be received from it. Last incompatible policy: RELIABILITY
```

La causa es una diferencia en la política de calidad de servicio (QoS) de ROS 2. MAVROS publica los topics de sensores con **BEST_EFFORT** (puede perder mensajes, menor latencia), pero el suscriptor por defecto de ROS 2 usa **RELIABLE** (exige entrega garantizada). DDS los considera incompatibles y descarta todos los mensajes.

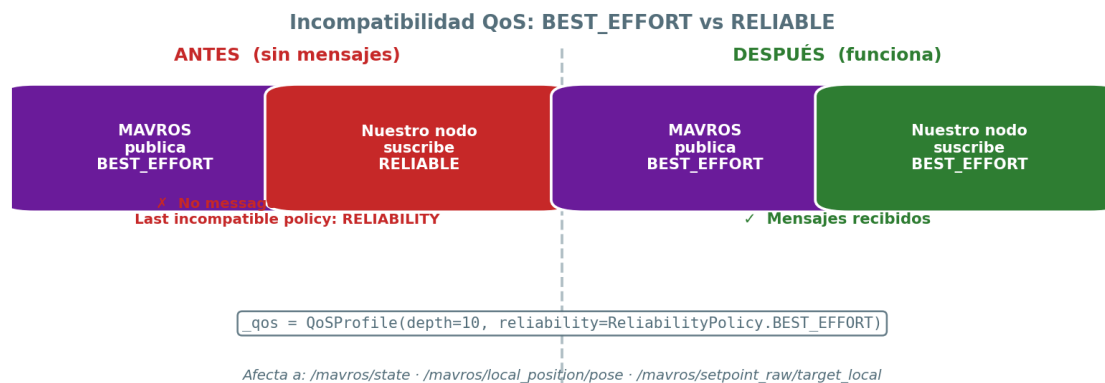


Figura 9 — Incompatibilidad QoS antes y después de la corrección.

Topics afectados y solución aplicada:

Topic MAVROS	QoS publicador	QoS original nuestro	QoS corregido
/mavros/state	BEST_EFFORT	RELIABLE	BEST_EFFORT
/mavros/local_position/pose	BEST_EFFORT	RELIABLE	BEST_EFFORT
/mavros/setpoint_raw/target_local	BEST_EFFORT	RELIABLE	BEST_EFFORT

La corrección se aplicó en **pixhawk_menu_node.py** y **mavros_node.py** creando un perfil QoS compartido:

```
from rclpy.qos import QoSProfile, ReliabilityPolicy _qos = QoSProfile(depth=10, reliability=ReliabilityPolicy.BEST_EFFORT) self.create_subscription(State, '/mavros/state', self._estado_cb, _qos) self.create_subscription(PoseStamped, '/mavros/local_position/pose', self._pose_cb, _qos)
```

15. Estado actual del proyecto

Componente	Estado	Detalle
USB Pixhawk → WSL2	✓ Operativo	usbipd-win, /dev/ttyACM0:57600, VID 3185:0038
MAVROS ↔ Pixhawk	✓ Conectado	Got HEARTBEAT, FCU: PX4 Autopilot
mavros_node → MAVROS	✓ Publicando	Publica en /mavros/setpoint_velocity/cmd_vel_unstamped
Posición real Pixhawk	■ Pendiente confirmar	Suscripción a /mavros/local_position/pose con QoS corregido
Estado MAVROS (armed/mode)	■ Pendiente confirmar	QoS BEST_EFFORT aplicado, pendiente reiniciar nodo
Round-trip [PX4 ACK]	■ Pendiente confirmar	Código en pixhawk_menu_node, necesita QoS fix activo
Simulador (simulador_dron)	✓ Sin cambios	Funciona en paralelo; mavros_node usa /drone/pose si no hay Pixhawk
cerebro_node → misión	✓ Sin cambios	Lee mision.json, secuencia de waypoints triple-Z

Próximos pasos para completar la integración:

- Reiniciar **pixhawk_menu_node** con el binario nuevo (QoS fix) y verificar que Modo ya no muestra DESCONOCIDO.
- Pulsar **[w]** y confirmar que aparece la línea [PX4 ACK] vx=+0.300 ... ✓.
- Con OFFBOARD activo, verificar que /mavros/local_position/pose actualiza la posición en el menú.
- Integrar **cerebro_node + mavros_node** con Pixhawk real para ejecutar la misión triple-Z en hardware.